

Introduction au développement Web

- J1 -



What is the Web?

Au cœur de son fonctionnement, le World Wide Web (souvent simplement appelé « le Web ») est un vaste système de documents et de ressources interconnectés, identifiés par des URL, reliés entre eux par des hyperliens et accessibles via Internet. On peut le comparer à une **immense bibliothèque mondiale**, où chaque livre (page web) est lié à de nombreux autres, créant ainsi une gigantesque base de connaissances consultable.

Internet vs. Web

- **Internet** : le réseau complexe de routes, d'autoroutes et d'infrastructures reliant les ordinateurs dans le monde entier. Il s'agit de l'ossature physique : fils, câbles et routeurs.
- **Le Web** : le « trafic » circulant sur ces routes : sites web, vidéos, documents et applications. Internet est le mécanisme de transmission, tandis que le Web est le contenu transmis.

The Web Ecosystem



Comprendre le DNS

Tout comme vous pourriez chercher le numéro de téléphone d'une personne dans un annuaire, les ordinateurs utilisent ce que l'on appelle le **Système de Noms de Domaine (DNS)** pour se trouver sur Internet. Au lieu de numéros de téléphone, les ordinateurs communiquent à l'aide d'adresses numériques appelées **adresses IP** (par exemple, 192.168.1.1 ou 2001:0db8::1).

❏ Une **adresse IP** (*Internet Protocol*) est un identifiant numérique unique attribué à chaque appareil connecté à un réseau. Elle fonctionne comme une **adresse postale** mais pour les ordinateurs : elle indique où les données doivent être envoyées et d'où elles proviennent.

- **IPv4** : composée de 4 nombres séparés par des points (ex. 192.168.1.1).
- **IPv6** : plus récente, composée de 8 groupes de chiffres et de lettres séparés par des : (ex. 2001:0db8::1).

Imaginez devoir mémoriser l'adresse IP de chaque site que vous visitez ! Ce serait impossible. C'est là qu'intervient le **DNS**. Le DNS agit comme un service d'annuaire de l'Internet, traduisant les noms de domaine conviviaux pour les humains (comme www.google.com) en adresses IP lisibles par les machines (comme 142.250.190.78).

Considérez le DNS comme un immense annuaire téléphonique distribué pour tout Internet.

Lorsque vous tapez le nom d'un site web dans votre navigateur, votre ordinateur ne sait pas immédiatement où se trouve ce site.

Il demande d'abord à un serveur DNS : « Quelle est l'adresse IP de example.com ? ». Le serveur DNS la recherche ensuite et fournit la bonne adresse numérique, permettant ainsi à votre navigateur de se connecter au serveur approprié.

Comment fonctionne le DNS (version simplifiée)

1. **Vous entrez un nom de domaine** (par ex. *example.com*) dans votre navigateur.
2. **Votre ordinateur demande** à un résolveur DNS l'adresse IP correspondante.
3. **Le résolveur consulte** différents serveurs DNS (racine → TLD → autoritaire) si nécessaire.
4. **L'adresse IP correcte est trouvée** et renvoyée à votre ordinateur.
5. **Votre navigateur utilise cette adresse IP** pour se connecter au serveur web et afficher le site.

Pourquoi le DNS est-il important ?

Sans le DNS, naviguer sur le web serait un véritable cauchemar. Nous devrions mémoriser de longues séries de chiffres pour chaque site.

Le DNS garantit que nos noms de domaine conviviaux sont traduits de manière transparente en adresses IP compréhensibles par les ordinateurs, rendant ainsi Internet accessible et simple à utiliser.

Un DNS défaillant peut entraîner des erreurs du type « *serveur introuvable* », même si le site web lui-même fonctionne correctement.

Language Web: HTTP Fundamentals

Une fois que votre navigateur connaît l'adresse IP du serveur, il a besoin d'un moyen de communiquer avec lui. C'est là qu'intervient **HTTP (Hypertext Transfer Protocol)**. HTTP est la base de la communication de données sur le World Wide Web : il définit comment les messages sont formatés et transmis, ainsi que les actions que les serveurs web et les navigateurs doivent entreprendre en réponse aux différentes commandes.

Considérez HTTP comme un ensemble de **règles d'échange** pour la communication client-serveur. C'est le langage qu'ils utilisent pour demander et fournir du contenu web.

HTTP Request

orsque vous cliquez sur un lien, tapez une URL ou soumettez un formulaire, votre navigateur envoie une **requête HTTP** au serveur. Cette requête comprend généralement :

- **Méthode** : l'action que vous souhaitez effectuer (par ex. GET pour récupérer des données, POST pour en envoyer).
- **URL** : la ressource spécifique que vous voulez.
- **En-têtes** : des métadonnées sur la requête (par ex. le type de navigateur, la langue préférée).
- **Corps (optionnel)** : les données envoyées (par ex. les informations d'un formulaire).

C'est comme passer une commande au restaurant : « Apportez-moi la pizza du menu », avec des précisions telles que « Je suis à la table 5 et je préfère l'italien ».

HTTP Response

Après avoir reçu et traité la requête, le serveur renvoie une **réponse HTTP**. Cette réponse contient :

- **Code de statut** : indique le succès ou l'échec de la requête (nous y reviendrons plus bas).
- **En-têtes** : des métadonnées sur la réponse (par ex. le type de contenu, les instructions de mise en cache).
- **Corps** : les données demandées (par ex. contenu HTML, fichier image, données JSON).

C'est comme le serveur au restaurant qui vous apporte votre plat : « Voici votre pizza (Corps) avec un sourire (Code de statut) et l'addition (En-têtes) ».

HTTP Status Codes

Les **codes de statut** sont des nombres à trois chiffres qui indiquent au client le résultat de sa requête. Ils sont essentiels pour le débogage et pour comprendre le comportement du serveur.

200 OK La requête a été effectuée avec succès et le serveur a renvoyé les données demandées. Tout est en ordre ! ✓	404 Not Found Le serveur n'a pas pu trouver la ressource demandée. C'est une erreur courante, souvent causée par un lien brisé ou une URL mal saisie. La page n'existe tout simplement pas à cette adresse.	500 Internal Server Error Un message d'erreur générique indiquant qu'un problème est survenu du côté du serveur. Cela signifie que le serveur a rencontré une condition inattendue qui l'a empêché de répondre à la requête. Ce type d'erreur nécessite souvent une investigation de la part du propriétaire du site web.
---	---	---

Web Request Lifecycle (Part 1)

Comprendre le parcours qu'effectue une requête web depuis votre ordinateur jusqu'à un serveur, puis en retour, est essentiel pour saisir le véritable fonctionnement du Web. Ce processus en plusieurs étapes, souvent réalisé en quelques millisecondes, implique plusieurs composants clés travaillant en harmonie.

Traçons ensemble le chemin d'une requête web typique, depuis le moment où vous appuyez sur « **Entrée** » jusqu'aux premières étapes de la réception du contenu

Step-by-Step: The Initial Handshake

01

1. L'utilisateur initie la requête

Tout commence lorsque vous, l'utilisateur, saisissez une URL (par ex. www.example.com) dans la barre d'adresse de votre navigateur et appuyez sur **Entrée**. Cette action déclenche le processus par lequel le navigateur tente de récupérer la page web souhaitée.

02

2. Le navigateur interroge le DNS

Votre navigateur doit d'abord trouver l'adresse IP de www.example.com. Il vérifie d'abord son cache DNS local, puis celui du système d'exploitation. Si l'adresse n'est pas trouvée, il envoie une requête à votre résolveur DNS configuré (souvent le serveur DNS de votre fournisseur d'accès Internet) en demandant : « *Quelle est l'adresse IP de www.example.com ?* »

03

3. Le DNS résout l'adresse IP

Le résolveur DNS effectue alors une série de recherches (pouvant impliquer plusieurs serveurs DNS à travers le monde) afin de trouver le serveur de noms autoritaire pour *example.com*. Ce serveur détient l'adresse IP réelle. Une fois trouvée, le résolveur DNS renvoie cette adresse IP à votre navigateur. Votre navigateur connaît désormais l'adresse numérique exacte du serveur avec lequel il doit communiquer.

A Web Request Lifecycle (Part 2)

Avec l'adresse IP en main, votre navigateur est désormais prêt à communiquer directement avec le serveur web. Les prochaines étapes consistent en l'échange réel d'informations et dans le rendu de la page web afin que vous puissiez la voir.

Cette partie du parcours met en évidence le rôle crucial du **protocole HTTP** et la manière dont différents éléments web s'assemblent pour offrir une expérience complète.

Step-by-Step: From Server to Screen

01

4. Le navigateur envoie une requête HTTP

Avec l'adresse IP en main, votre navigateur ouvre une connexion vers le serveur web et envoie une requête HTTP GET pour la page `/index.html` (ou toute autre page principale). Cette requête inclut généralement des informations sur votre navigateur, vos langues préférées ainsi que d'autres en-têtes nécessaires.

02

5. Le serveur traite la requête

Le serveur web reçoit la requête HTTP. Il localise la ressource demandée (*index.html* dans ce cas). Si la page nécessite du contenu dynamique (par ex. profils utilisateurs, listes de produits), le serveur peut interagir avec des applications backend ou des bases de données pour récupérer les données pertinentes et assembler le document HTML complet.

03

6. Le serveur envoie une réponse HTTP

Une fois la page prête, le serveur renvoie une réponse HTTP à votre navigateur. Cette réponse contient un code de statut HTTP (idéalement **200 OK**) ainsi que le contenu réel de la page web, principalement un document HTML.

04

7. Le navigateur affiche la page

Votre navigateur reçoit le fichier HTML et commence à l'analyser ligne par ligne. Lorsqu'il rencontre des références à d'autres ressources (comme des feuilles de style CSS, des fichiers JavaScript, des images ou des vidéos), il envoie des requêtes HTTP supplémentaires au serveur pour les récupérer. Une fois toutes les ressources téléchargées, le navigateur les combine, applique les styles et exécute les scripts, affichant enfin la page web entièrement rendue à l'écran.

Cette séquence complète — depuis la saisie de l'URL jusqu'à l'affichage de la page — se produit des milliers de fois par jour et constitue l'ossature de votre expérience interactive sur le Web. Comprendre ces étapes vous permet non seulement de résoudre plus facilement les problèmes, mais aussi de concevoir des applications web plus efficaces.

Les blocs de construction du Web : HTML

Le premier langage que vous rencontrerez dans votre parcours de développement web est **HTML**, qui signifie *HyperText Markup Language*. C'est le langage fondamental pour créer des pages web : il fournit la **structure** et le **contenu** qui constituent la colonne vertébrale de chaque site que vous visitez.

Que fait HTML ?

1

Définit la structure

HTML utilise une série d'éléments (ou balises) pour définir les différentes parties d'une page web. Ces éléments indiquent au navigateur comment organiser le contenu, comme les titres, les paragraphes, les listes, les liens, les images et les tableaux.

👉 Sans HTML, une page web ne serait qu'un désordre de texte brut, sans structure ni hiérarchie.

2

Organise le contenu

HTML définit la **hiérarchie** et les **relations** entre les différentes parties d'une page.

- Par exemple, une balise `<h1>` indique le titre le plus important de la page.
- Une balise `<p>` sert à représenter un paragraphe de texte.

👉 Cette structure sémantique est essentielle, à la fois pour la **lisibilité par les humains** et pour **l'interprétation par les moteurs de recherche**.

3

Hyperlinks !!

Le terme « **HyperText** » dans HTML fait référence à sa capacité à relier des documents entre eux. La balise `<a>` (appelée *ancree*) est utilisée pour créer des hyperliens, permettant aux utilisateurs de naviguer d'une page web à une autre.

👉 C'est ce mécanisme qui a donné naissance au Web interconnecté que nous connaissons aujourd'hui.

Exemple

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Welcome to My Website!</h1>
  <p>This is my very first web page created with HTML.
    It's amazing how simple it is to structure content.</p>
  <a href="https://www.example.com">Visit Example.com</a>
</body>
</html>
```

- `<!DOCTYPE html>` : informe le navigateur que le document est en **HTML5**.
- `<html lang="fr">` : balise racine du document ; l'attribut `lang="fr"` indique que la langue principale est le français.
- `<head>` : contient les **métadonnées** (informations sur la page, non affichées directement).
- `<meta charset="UTF-8">` : définit l'encodage des caractères (UTF-8 → permet d'afficher correctement les accents).
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` : assure un **affichage adapté sur mobile** (responsive design).
- `<title>` : définit le **titre de l'onglet** dans le navigateur.
- `<body>` : contient le **contenu visible** de la page (textes, images, liens...).
- `<h1>` : titre principal de la page.
- `<p>` : paragraphe de texte.
- `<a href="URL">` : crée un **lien hypertexte**.
 - `href="https://www.google.com"` → destination du lien.
 - Texte à l'intérieur → ce qui est **cliquable**.
- `target="_blank"` (dans un lien) → ouvre le lien dans un **nouvel onglet**.
- Lien interne** (`about.html`) → mène à une page locale du même site.
- `</html>` : fin du document HTML.

Styler le Web : CSS

Alors que **HTML** fournit la structure d'une page web, **CSS** (*Cascading Style Sheets* ou feuilles de style en cascade) est ce qui lui donne vie visuellement.

CSS est un langage puissant utilisé pour décrire la **présentation** d'un document écrit en HTML.

Il définit comment les éléments HTML doivent être affichés à l'écran, sur papier ou dans d'autres supports.

What Does CSS Control?

Esthétique visuelle

CSS est responsable des couleurs, des polices, des tailles de texte, des images d'arrière-plan et bien plus encore.

Il permet aux concepteurs d'appliquer un **thème visuel cohérent** à l'ensemble d'un site web, garantissant une apparence professionnelle et fidèle à l'identité de la marque.

Mise en page et positionnement

Au-delà des couleurs, **CSS** contrôle l'**agencement des éléments** sur la page.

Cela inclut :

- les **marges** (*margins*),
- les **espacements internes** (*padding*),
- la **largeur et la hauteur** des éléments,
- ainsi que des techniques avancées comme **Flexbox** et **CSS Grid**,

👉 Ces outils permettent de créer des **designs responsives**, capables de s'adapter automatiquement aux différentes tailles d'écrans (ordinateur, tablette, smartphone).

Responsivité

Avec **CSS**, les sites web peuvent être conçus pour avoir un rendu optimal sur tout type d'appareil, qu'il s'agisse de grands écrans d'ordinateur ou de petits téléphones mobiles.

Les **media queries** en CSS permettent d'appliquer des styles différents selon la **largeur de l'écran**, la **résolution** ou d'autres caractéristiques.

👉 Cela assure une expérience utilisateur fluide et adaptée, peu importe le support utilisé.

CSS Example

```
body {
  background-color: #F0F8FF; /* Alice Blue */
  font-family: Arial, sans-serif;
  margin: 20px;
  padding: 20px;
}

h1 {
  color: #0056b3; /* Dark Blue */
  text-align: center;
  text-shadow: 2px 2px 4px #cccccc;
}

p {
  font-size: 1.1em;
  line-height: 1.6;
  color: #333333;
}

a {
  color: #007bff;
  text-decoration: none;
  font-weight: bold;
}

a:hover {
  text-decoration: underline;
  color: #0056b3;
}
```

JavaScript

Après que **HTML** a fourni la structure et que **CSS** a ajouté le style, **JavaScript (JS)** est le langage de programmation qui rend les pages web **interactives et dynamiques**.
C'est le « **cerveau** » **du web**, permettant de créer des fonctionnalités complexes, des animations et des mises à jour en temps réel que HTML et CSS, seuls, ne peuvent pas réaliser.

Que fait JavaScript ?



Définit le comportement

JavaScript gère les **interactions de l'utilisateur**, comme les clics, la soumission de formulaires ou la saisie au clavier.
Il permet aux pages web de **réagir à ces actions** sans nécessiter un rechargement complet de la page.



Contenu dynamique

JavaScript peut :

- modifier le **HTML** et le **CSS** à la volée,
- mettre à jour du contenu de façon **asynchrone** (par ex. récupérer des données depuis un serveur sans recharger la page),
- créer des **éléments animés**,
- et **valider les données d'un formulaire** avant son envoi.

👉 C'est ce qui rend une page web vivante et interactive.



Communication avec le serveur

Grâce à des technologies comme **AJAX** (*Asynchronous JavaScript and XML*) ou l'**API Fetch**, **JavaScript** permet aux pages web de **communiquer avec les serveurs en arrière-plan**.

- Il peut récupérer de nouvelles données,
- ou envoyer des informations saisies par l'utilisateur, 👉 le tout **sans interrompre l'expérience** ni nécessiter de rechargement complet de la page.

JavaScript Example

Ce code JavaScript affiche une **alerte** lorsque la page se charge et modifie le **texte d'un paragraphe** lorsqu'un bouton est cliqué.

```
// JavaScript often runs when the page is loaded
window.onload = function() {
  alert("Hello, future developer! Welcome to my dynamic page!");
};

// Function to change text on button click
function changeText() {
  const paragraph = document.getElementById("myParagraph");
  if (paragraph) {
    paragraph.textContent = "You clicked the button! JavaScript is awesome!";
    paragraph.style.color = "#0056b3"; // Change text color with JS
  }
}

// In HTML, you'd have:
// <p id="myParagraph">Click the button below!</p>
// <button onclick="changeText()">Click Me</button>
```

Cet exemple illustre la capacité de **JavaScript** à interagir avec le **Document Object Model (DOM)** – la représentation programmatique du document HTML.

- Il peut **manipuler les éléments**,
 - **modifier les styles**,
 - et **déclencher des actions** en fonction des interactions de l'utilisateur,
- 👉 transformant ainsi les pages web en véritables **applications interactives**.

Test Your Knowledge: A Quick Web Quiz!

Now that we've covered the fundamental concepts of the Internet, the Web, and its core languages, let's put your newfound knowledge to the test! This mini-quiz will help solidify your understanding and highlight any areas you might want to review. Don't worry if you don't get all the answers right – the goal is to learn and reinforce what you've just discovered.

Challenge Yourself!

“

Question 1:

Laquelle des options suivantes définit la **structure d'une page web**, y compris les titres, les paragraphes et les liens ?

- A) CSS
- B) HTML
- C) JavaScript
- D) DNS

“

Question 2:

Si vous recevez un code de statut HTTP **404 Not Found**, que signifie-t-il généralement ?

- A) La requête a réussi.
- B) Le serveur n'a pas pu trouver la ressource demandée.
- C) Il y a une erreur interne du serveur.
- D) La connexion réseau est interrompue.

”

”

“

Question 3:

Quel langage est principalement responsable d'ajouter de l'**interactivité** et un **comportement dynamique** à une page web ?

- A) HTML
- B) CSS
- C) JavaScript
- D) HTTP

“

Question 4:

Quelle est la fonction principale du **Système de Noms de Domaine (DNS)** ?

- A) Mettre en forme les pages web.
- B) Gérer la logique côté serveur.
- C) Traduire les noms de domaine en adresses IP.
- D) Définir des animations interactives.

”

”

Answers : B - B - C - C

Vos premiers pas dans le développement web

résumé et perspectives

 **Félicitations !** Vous venez de terminer une introduction intensive aux concepts fondamentaux des technologies web. De la compréhension de la différence entre **Internet et le Web**, à l'exploration du **cycle de vie d'une requête web**, en passant par les langages essentiels qui construisent le monde numérique, vous avez acquis des connaissances précieuses. Ces bases sont essentielles pour commencer votre parcours vers le métier de développeur web compétent.

Points clés de la journée 1

Le Web est **un système**
C'est une interaction complexe entre clients, serveurs et réseaux, orchestrée par des protocoles comme HTTP et DNS pour délivrer du contenu à l'échelle mondiale.

La communication est **essentielle**
HTTP et DNS sont les rouages silencieux qui permettent aux navigateurs et aux serveurs web de « parler » et de comprendre les requêtes et réponses.

Trois langages construisent l'expérience

- **HTML** pour la structure
- **CSS** pour le style
- **JavaScript** pour l'interactivité
Ensemble, ils donnent vie aux sites riches et dynamiques que nous utilisons chaque jour.

- **Pratiquer HTML & CSS** : créez vos premières pages web statiques. Testez différentes balises HTML et appliquez des propriétés CSS pour voir leurs effets.
- **Découvrir la logique de base en JavaScript** : travaillez sur les variables, fonctions et manipulations simples du DOM. Faites apparaître et disparaître des éléments avec un bouton.
- **Explorer les outils de développement** : allez plus loin que l'onglet *Network*. Inspectez les éléments, modifiez les styles à la volée, et déboguez du JavaScript.
- **Choisir un éditeur de texte** : optez pour un bon éditeur (comme **VS Code**) avec coloration syntaxique et autocomplétion pour améliorer votre expérience de codage.

Afternoon

1. Setup VS Code (20 min)

- Show how to install VS Code.
- Install extension **Live Server**.
- Create a folder my-first-site.
- Inside: create index.html.

2. First HTML Page (30 min)

Write together:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

3. Explore HTML Elements (60 min)

- **Headings:** <h1> to <h6>.
- **Paragraphs:** <p>.
- **Links:** Visit Example.
- **Images:** .
- **Lists:**
 - Ordered Step 1.
 - Unordered Apple.

💡 **Exercise 1:** Create a page with:

- A title <h1> with their name.
- A short description in <p>.
- A list of hobbies.
- A link to their favorite website.

Instructions:

- Title: their name in <h1> .
- Subtitle <h2> : “Web Developer in training” (or similar).
- Profile image using placeholder.
- Section *About Me* → one paragraph.
- Section *Skills* → list (HTML, CSS, JavaScript).
- Section *Hobbies* → list.
- Section *Contact* → link to email (`mailto:`) or GitHub/LinkedIn.

💡 **Bonus:** encourage creativity (colors, emojis, favorite quote).

5. Wrap-Up & Homework (20 min)

- Improve the CV page → add more sections.
- Prepare a question for tomorrow about CSS.